

# SyncArena — Motor de Juegos Multijugador

Aporte técnico: Patrón de diseño **Singleton** (versión JavaScript / Node.js) aplicado a la gestión de partidas, jugadores y ranking

---

|                         |                                                                   |
|-------------------------|-------------------------------------------------------------------|
| <b>Proyecto:</b>        | SyncArena                                                         |
| <b>Alumnos:</b>         | Oscar Julian Gamboa Sanabria — Jesus Javier Garcia Rojas          |
| <b>Módulo:</b>          | Gestión de partidas, jugadores y rankings                         |
| <b>Patrón aportado:</b> | Singleton (vía closure + clase)                                   |
| <b>Lenguaje:</b>        | JavaScript (Node.js 22)                                           |
| <b>Pruebas:</b>         | node:test (test runner nativo de Node, sin dependencias externas) |

## 1. ¿Qué es el patrón Singleton?

Singleton es un patrón de diseño creacional que garantiza que una clase tenga **una única instancia** durante toda la ejecución del programa, y ofrece un punto de acceso global a esa instancia. Cualquier parte del sistema que pida un objeto de esa clase recibe siempre la misma referencia, en lugar de crear copias independientes.

## 2. ¿Por qué aporta valor a SyncArena?

El objetivo central del proyecto es mantener sincronizado el estado del juego entre todos los clientes conectados. Eso solo es posible si existe **un único punto de verdad** para ese estado. Si el módulo de matchmaking, el de chat y el de ranking crearan cada uno su propia copia del gestor de partidas, el estado se desincronizaría entre jugadores — exactamente el problema que el proyecto busca resolver.

El aporte concreto de este documento es la clase `GameServerManager`, responsable del pilar 1 ("Gestión de partidas, jugadores y rankings"). Se implementa como Singleton mediante una **función de orden superior reutilizable** (`crearSingleton`), de forma que otros gestores del motor (conexión de red, configuración del servidor, logging) puedan adoptar el mismo mecanismo sin duplicar código.

### Nota sobre concurrencia en JavaScript

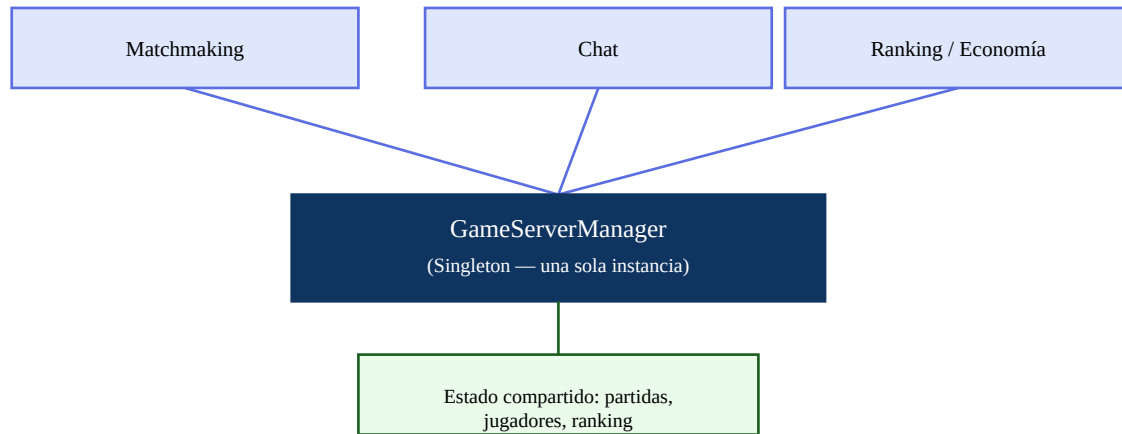
A diferencia de una versión en un lenguaje multihilo, Node.js ejecuta el código en un único hilo con un event loop: dos llamadas a `new GameServerManager()` nunca se ejecutan literalmente al mismo tiempo, así que no hace falta un mecanismo de bloqueo ("lock"). Aun así, se probó el escenario con 50 creaciones lanzadas de forma concurrente vía `Promise.all` para confirmar que todas resuelven a la misma instancia, incluso cuando hay trabajo asíncrono de por medio.

### Consideración honesta

Singleton es un patrón útil pero también criticado: si se abusa de él, genera estado global oculto y dificulta las pruebas unitarias. Por eso esta implementación incluye un método estático `resetInstance()` pensado solo para pruebas, y se recomienda reservar el patrón para gestores realmente únicos por servidor (partidas,

conexiones, configuración), no para cualquier clase del motor.

### 3. Diagrama del mecanismo



Los tres módulos piden cada uno su propia referencia con `new GameServerManager()`, pero el closure del Singleton hace que todos reciban el mismo objeto en memoria y, por lo tanto, vean siempre el mismo estado.

## 4. Implementación

### 4.1 Utilidad Singleton reutilizable — singleton.js

```
/**
 * singleton.js
 * -----
 * Utilidad genérica para aplicar el patrón Singleton a cualquier clase
 * del motor SyncArena (GameServerManager, ConnectionPool, ConfigManager,
 * Logger, etc), sin repetir la lógica de "una sola instancia" en cada una.
 *
 * Nota sobre concurrencia en JS:
 * A diferencia de Python, JavaScript (Node.js) ejecuta el código en un
 * solo hilo con un event loop. Por eso aquí no hace falta un "lock" como
 * en la versión Python: no pueden existir dos llamadas a getInstance()
 * ejecutándose en el mismo instante. Aun así, esta utilidad se probó
 * lanzando muchas creaciones concurrentes (vía Promise.all) para
 * confirmar que todas resuelven a la misma instancia.
 */

function crearSingleton(ClaseBase) {
  let instancia = null;

  return class extends ClaseBase {
    constructor(...args) {
      if (instancia) {
        // Si ya existe una instancia, la devolvemos en vez de crear otra.
        // eslint-disable-next-line no-constructor-return
        return instancia;
      }
      super(...args);
      instancia = this;
    }
  };

  /**
   * Utilidad SOLO para pruebas: permite "olvidar" la instancia guardada
   * para poder probar la clase desde cero en cada test. No debería
   * usarse en código de producción.
   */
  static resetInstance() {
    instancia = null;
  }
};

module.exports = { crearSingleton };
```

### 4.2 Aplicación concreta — gameServerManager.js

```
/**
 * gameServerManager.js
 * -----
 * Aporte concreto al pilar "Gestión de partidas, jugadores y rankings"
 * de SyncArena, usando el patrón Singleton.
 *
 * Por qué Singleton aquí:
 * En un motor multijugador debe existir UN solo punto de verdad que sepa
 * qué partidas están activas, qué jugadores están conectados y cuál es
 * el ranking global. Si cada módulo (chat, matchmaking, economía) creara
 * su propia copia de este gestor, el estado se desincronizaría entre
 * jugadores -> justo el problema que SyncArena busca evitar.
 */

const { crearSingleton } = require("../singleton");
class Partida {
```

```

    constructor(idPartida) {
        this.idPartida = idPartida;
        this.jugadores = [];
        this.estado = "esperando"; // esperando | en_curso | finalizada
    }
}

class GameServerManagerBase {
    constructor() {
        this._partidas = new Map();
        this._jugadoresConectados = [];
        this._ranking = new Map();
    }

    // ----- Gestión de partidas -----

    crearPartida(idPartida) {
        if (this._partidas.has(idPartida)) {
            throw new Error(`La partida '${idPartida}' ya existe.`);
        }
        const partida = new Partida(idPartida);
        this._partidas.set(idPartida, partida);
        return partida;
    }

    unirJugadorAPartida(idPartida, jugador) {
        const partida = this._partidas.get(idPartida);
        if (!partida) {
            throw new Error(`La partida '${idPartida}' no existe.`);
        }
        if (!partida.jugadores.includes(jugador)) {
            partida.jugadores.push(jugador);
        }
    }

    finalizarPartida(idPartida) {
        const partida = this._partidas.get(idPartida);
        if (!partida) {
            throw new Error(`La partida '${idPartida}' no existe.`);
        }
        partida.estado = "finalizada";
    }

    obtenerPartida(idPartida) {
        return this._partidas.get(idPartida);
    }

    // ----- Gestión de jugadores conectados -----

    conectarJugador(jugador) {
        if (!this._jugadoresConectados.includes(jugador)) {
            this._jugadoresConectados.push(jugador);
        }
        if (!this._ranking.has(jugador)) {
            this._ranking.set(jugador, 0);
        }
    }

    desconectarJugador(jugador) {
        this._jugadoresConectados = this._jugadoresConectados.filter(
            (j) => j !== jugador
        );
    }

    jugadoresConectados() {
        return [...this._jugadoresConectados];
    }
}

```

```
// ----- Ranking -----

sumarPuntos(jugador, puntos) {
  const actual = this._ranking.get(jugador) || 0;
  this._ranking.set(jugador, actual + puntos);
}

obtenerRanking() {
  return [...this._ranking.entries()].sort((a, b) => b[1] - a[1]);
}
}

// Aplicamos el Singleton genérico sobre la clase base
const GameServerManager = crearSingleton(GameServerManagerBase);

module.exports = { GameServerManager };
```

## 5. Pruebas de funcionamiento

Se implementaron 8 pruebas con el test runner nativo de Node (node:test + assert, sin dependencias externas como Jest), divididas en dos grupos: (a) pruebas que verifican el comportamiento del propio patrón Singleton (misma instancia, estado compartido, creaciones concurrentes) y (b) pruebas que verifican la lógica de negocio del gestor (partidas, jugadores, ranking).

### 5.1 Código de pruebas — gameServerManager.test.js

```
/**
 * gameServerManager.test.js
 * -----
 * Pruebas de funcionamiento para el aporte Singleton de SyncArena en JS.
 * Usa el test runner nativo de Node (node:test) + assert, sin dependencias
 * externas.
 *
 * Ejecutar con: node --test
 */

const test = require("node:test");
const assert = require("node:assert/strict");
const { GameServerManager } = require("../gameServerManager");

// ----- Comportamiento del patrón Singleton -----

test("misma instancia en llamadas repetidas", () => {
  GameServerManager.resetInstance();
  const instancia1 = new GameServerManager();
  const instancia2 = new GameServerManager();
  assert.strictEqual(instancia1, instancia2);
});

test("estado compartido entre referencias", () => {
  GameServerManager.resetInstance();
  const instancia1 = new GameServerManager();
  instancia1.conectarJugador("Oscar");

  const instancia2 = new GameServerManager(); // "otra" referencia
  assert.ok(instancia2.jugadoresConectados().includes("Oscar"));
});

test("una sola instancia con creaciones concurrentes (Promise.all)", async () => {
  GameServerManager.resetInstance();

  const crearInstanciaAsync = () =>
    new Promise((resolve) => {
      // Simula trabajo asíncrono antes de crear la instancia
      setTimeout(() => resolve(new GameServerManager()), 100);
    });

  const instancias = await Promise.all(
    Array.from({ length: 50 }, () => crearInstanciaAsync())
  );

  const referenciaUnica = instancias[0];
  const todasIguales = instancias.every((i) => i === referenciaUnica);
  assert.ok(todasIguales, "Se crearon múltiples instancias del Singleton");
});

// ----- Lógica de negocio del GameServerManager -----

test("crear partida y unir jugadores", () => {
  GameServerManager.resetInstance();
  const gestor = new GameServerManager();
```

```

gestor.crearPartida("partida_1");
gestor.unirJugadorAPartida("partida_1", "Oscar");
gestor.unirJugadorAPartida("partida_1", "Jesus");

const partida = gestor.obtenerPartida("partida_1");
assert.deepStrictEqual(partida.jugadores, ["Oscar", "Jesus"]);
assert.strictEqual(partida.estado, "esperando");
});

test("no permite partidas duplicadas", () => {
  GameServerManager.resetInstance();
  const gestor = new GameServerManager();

  gestor.crearPartida("partida_1");
  assert.throws(() => gestor.crearPartida("partida_1"), /ya existe/);
});

test("finalizar partida", () => {
  GameServerManager.resetInstance();
  const gestor = new GameServerManager();

  gestor.crearPartida("partida_1");
  gestor.finalizarPartida("partida_1");
  assert.strictEqual(gestor.obtenerPartida("partida_1").estado, "finalizada");
});

test("conectar y desconectar jugador", () => {
  GameServerManager.resetInstance();
  const gestor = new GameServerManager();

  gestor.conectarJugador("Oscar");
  assert.ok(gestor.jugadoresConectados().includes("Oscar"));

  gestor.desconectarJugador("Oscar");
  assert.ok(!gestor.jugadoresConectados().includes("Oscar"));
});

test("ranking se ordena de mayor a menor", () => {
  GameServerManager.resetInstance();
  const gestor = new GameServerManager();

  gestor.conectarJugador("Oscar");
  gestor.conectarJugador("Jesus");

  gestor.sumarPuntos("Oscar", 50);
  gestor.sumarPuntos("Jesus", 120);
  gestor.sumarPuntos("Oscar", 30); // Oscar queda con 80

  const ranking = gestor.obtenerRanking();
  assert.deepStrictEqual(ranking[0], ["Jesus", 120]);
  assert.deepStrictEqual(ranking[1], ["Oscar", 80]);
});

```

## 5.2 Resultado real de la ejecución (node --test)

```

TAP version 13
# Subtest: misma instancia en llamadas repetidas
ok 1 - misma instancia en llamadas repetidas
---
  duration_ms: 1.838331
  type: 'test'
  ...
# Subtest: estado compartido entre referencias
ok 2 - estado compartido entre referencias
---
  duration_ms: 0.446571

```

```

    type: 'test'
    ...
# Subtest: una sola instancia con creaciones concurrentes (Promise.all)
ok 3 - una sola instancia con creaciones concurrentes (Promise.all)
---
    duration_ms: 15.604805
    type: 'test'
    ...
# Subtest: crear partida y unir jugadores
ok 4 - crear partida y unir jugadores
---
    duration_ms: 1.277557
    type: 'test'
    ...
# Subtest: no permite partidas duplicadas
ok 5 - no permite partidas duplicadas
---
    duration_ms: 0.565864
    type: 'test'
    ...
# Subtest: finalizar partida
ok 6 - finalizar partida
---
    duration_ms: 0.205417
    type: 'test'
    ...
# Subtest: conectar y desconectar jugador
ok 7 - conectar y desconectar jugador
---
    duration_ms: 3.395345
    type: 'test'
    ...
# Subtest: ranking se ordena de mayor a menor
ok 8 - ranking se ordena de mayor a menor
---
    duration_ms: 0.312419
    type: 'test'
    ...
1..8
# tests 8
# suites 0
# pass 8
# fail 0
# cancelled 0
# skipped 0
# todo 0
# duration_ms 128.646313

```

Las 8 pruebas pasaron correctamente (**8 pass, 0 fail**), incluyendo la prueba con 50 creaciones lanzadas de forma concurrente vía `Promise.all`, lo que confirma que la implementación resuelve siempre a la misma instancia — un requisito realista para un servidor de juego con miles de jugadores conectados.

## 6. Demostración de uso entre módulos

El script `demoUso.js` simula tres módulos distintos (matchmaking, chat y ranking) pidiendo cada uno su propia instancia de `GameServerManager`, sin pasársela por parámetro. La salida real confirma con `===` (comparación de referencia) que el objeto es el mismo antes y después, y que el módulo de chat ve los jugadores que agregó matchmaking sin que se le haya informado directamente.

```

¿instancia1 === nueva llamada? true
[chat] Jugadores en arena_01: [ 'OscarJulian', 'JesusJavier' ]
[ranking] Tabla actual: [ [ 'JesusJavier', 150 ], [ 'OscarJulian', 100 ] ]

```

```
¿instancia1 === instancia2 (tras usar los módulos)? true
```

## 7. Conclusión

El patrón Singleton aporta al proyecto SyncArena un mecanismo simple y probado para resolver uno de sus problemas centrales: mantener un único estado consistente de partidas, jugadores y ranking, accesible desde cualquier módulo del motor sin riesgo de duplicación ni desincronización. La utilidad implementada en JavaScript es reutilizable para futuros gestores del proyecto (conexiones de red, configuración del servidor, logging), manteniendo el principio de no reinventar la rueda que persigue SyncArena.